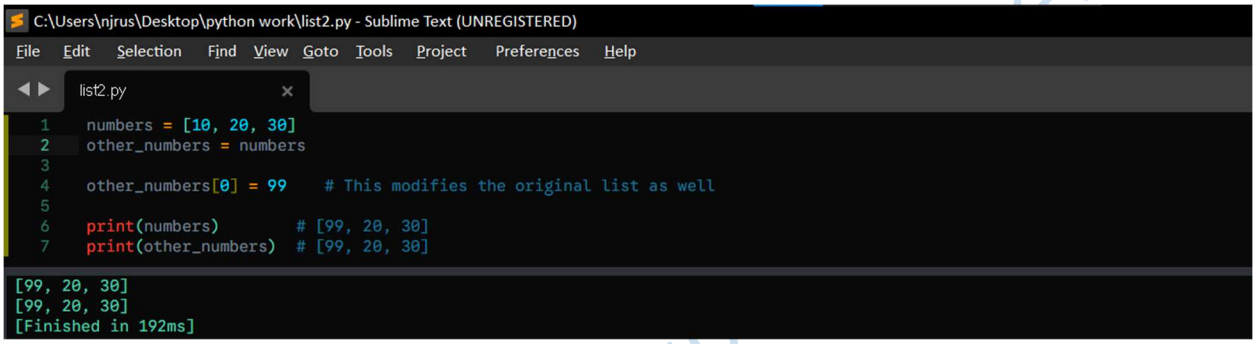
# More Concepts for Working with Lists

## 1. Copying Lists and References

Sometimes you want to keep one list unchanged while working with another list based on the same values. To avoid unexpected changes to your original data, you need to know how Python handles list references and how to create a proper copy.

### Assigning a list to another variable

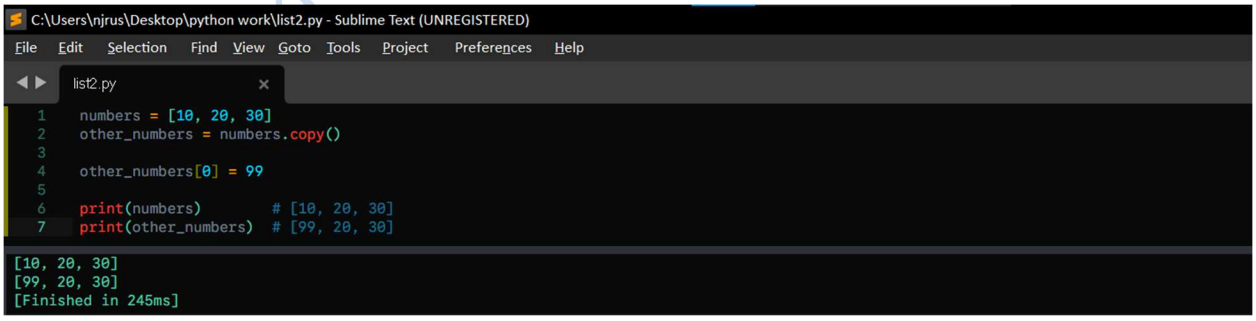
Assigning an existing list to a new variable does not automatically create a second list. Both variable names map to the same list in memory.




Note that we modified **other_numbers** and not the original list **numbers**. But since both variables refer to the same list, modifying other_numbers also affects what you see through numbers. Avoid using this style to copy your lists if you do not intend to modify your original list.

### Making a copy with .copy()

The copy() list method creates a separate, shallow copy. For lists containing simple values such as numbers or strings, changing an item in the copy does not change the corresponding item in the original.

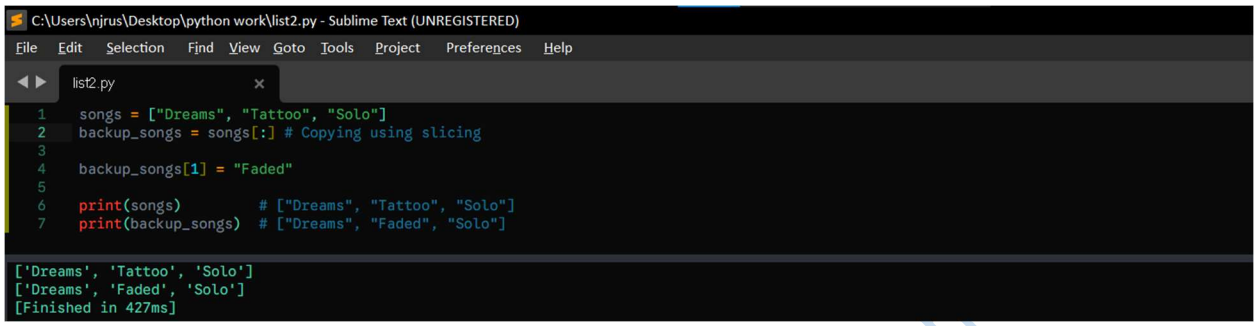



The original list remains intact while the copy gets modified. This is one of the recommended ways to copy your lists and some other data structures like **tuples, dictionaries** etc.

### Copying with slicing

List slicing can also be used to create list copies, as discussed in the previous lesson. Recall that, to create a copy using slicing, this syntax is used:

```
new_list_name = original_list[ : ]
```




Modifying the copied lists does not change items in the original list. For lists, **.copy()** & **[ : ]** are both convenient ways to make a separate list.

### Quick comparison

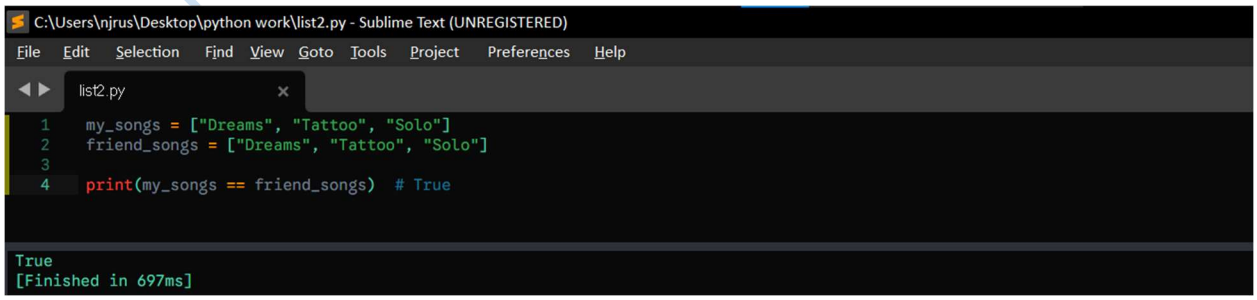
| Code | New list? | What happens? |
|---|---|---|
| **other = original** | No | Both names refer to the same list. |
| **other = original.copy()** | Yes | Creates a shallow copy. |
| **other = original[:]** | Yes | Creates a shallow copy. |

## 2. Comparing Lists and Membership

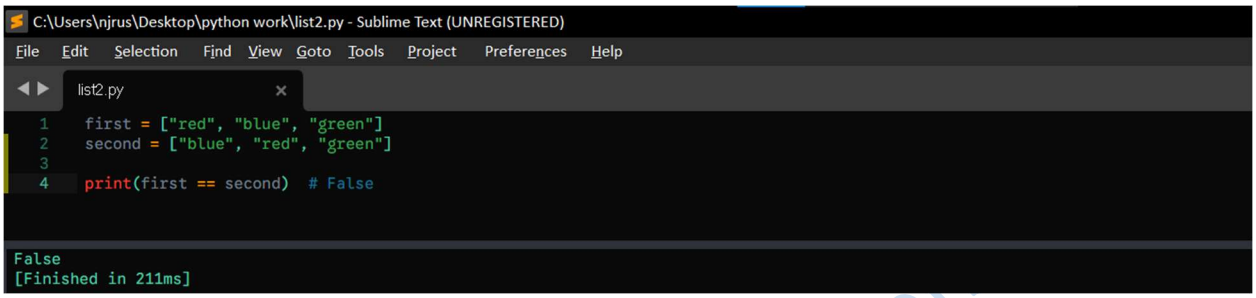
Lists let us keep related values together. As programs become more useful, we often need to answer questions about the data inside them: Do these two lists match? Is a user already registered? Is an item missing from an inventory? Python provides comparison and membership operators to help answer these questions.

### Comparing lists Using == and != Operators

Python lets you compare two separate lists by checking whether they contain the exact same values in the exact same order. The equality == and inequality != operators are used for this.



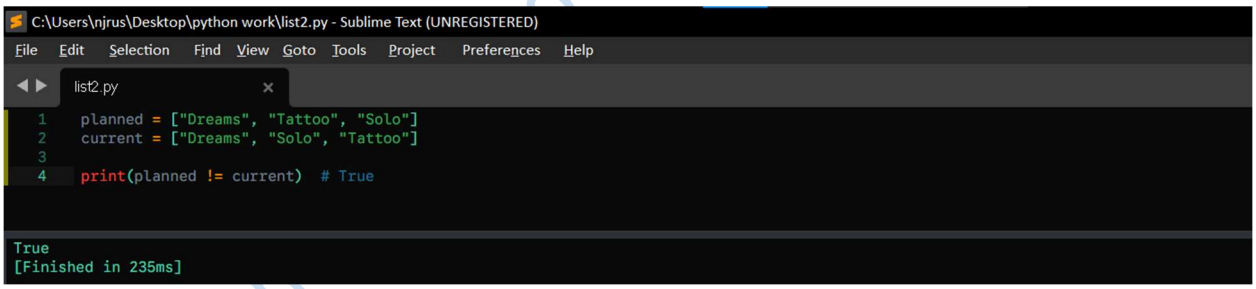
The equality operator (==) checks whether two lists contain the same items in the same order. It returns a Boolean value. That is, returns True if they compare equal, and False when they are not. The result in the above program is True because both lists contain the same values in the same positions/order: "Dreams" is first, "Tattoo" is second, and "Solo" is third.




Here, both lists contain "red", "blue", and "green", but the first two values have swapped positions. Since the lists do not match item by item in the same order, the result is False.

**Note: Python compares the order of items and contents of the lists to determine equality; the variables do not need to refer to the very same list object.**

The inequality operator (!=) checks whether two values are different. For lists, it returns True when their contents or order make them unequal, and False when they compare equal.




The lists differ in order, so != returns True.

The number of items matters as well. If one list has extra items, the lists cannot be equal.

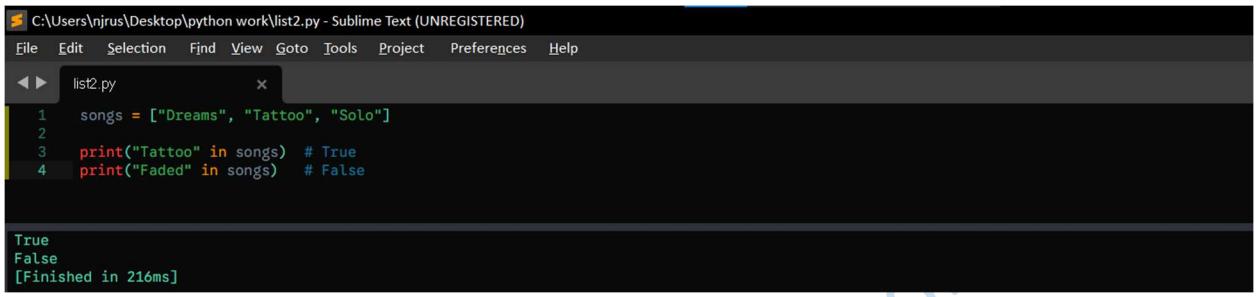



The first two items match, but long_list has an additional item. Therefore, the lists are not equal.

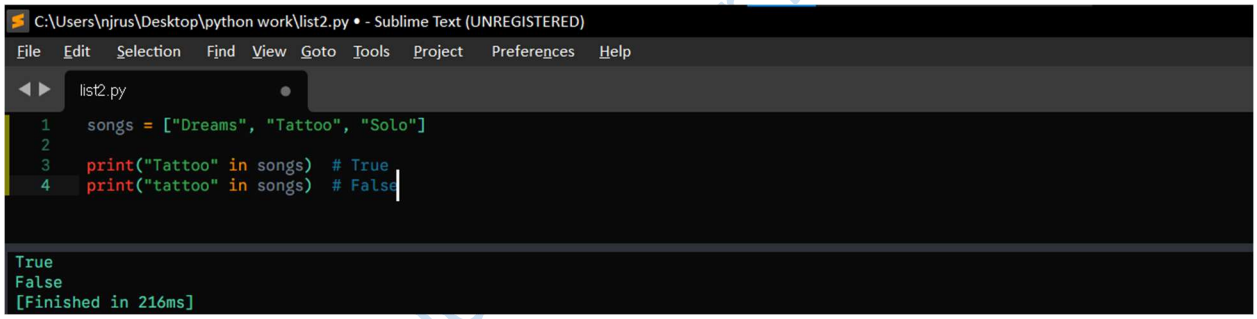
## Checking Membership in a List

Sometimes we do not need to compare two entire lists. We only want to know whether one particular value appears in a list. This is called a membership check.

The **in** operator is used in such circumstances to check whether a value is present in a list. It returns True if the value is found and False if it is not.

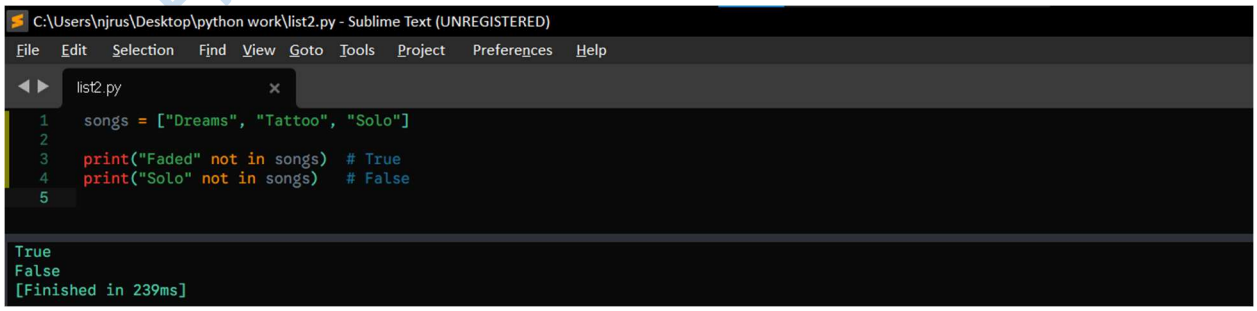



Note that membership checks compare exact values. For example, when checking strings, uppercase and lowercase letters are treated as different characters. For a membership check to succeed, the searched value must match the stored string.



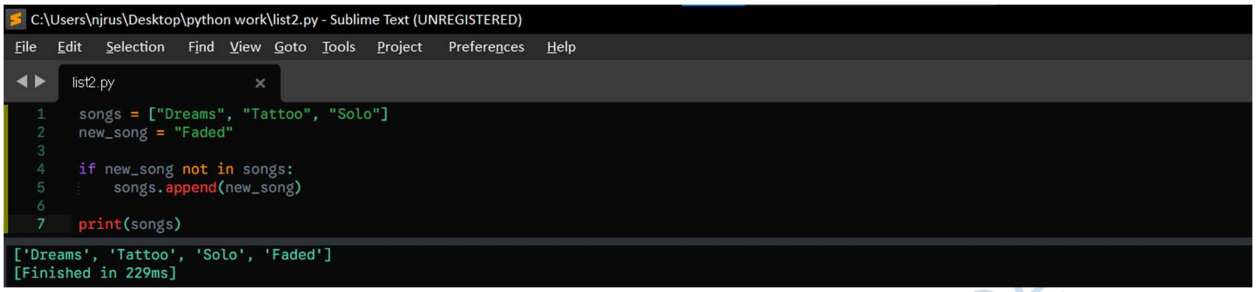

The second check is False because "tattoo" starts with a lowercase t, while the stored item "Tattoo" starts with an uppercase T. They are different strings.

The **not in** operator checks the opposite condition: whether a value is absent from a list. It returns True when the value is not present, and False when the value is present.

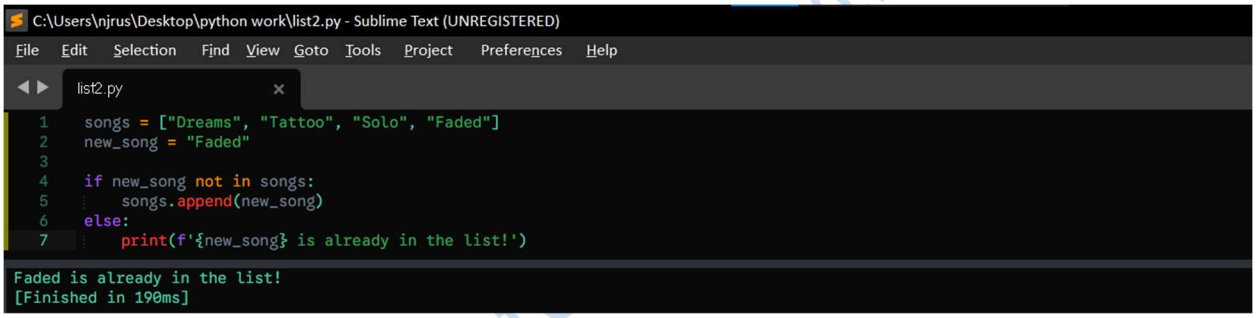



"Faded" is absent, so the first expression is True. "Solo" is present, so the second is False.

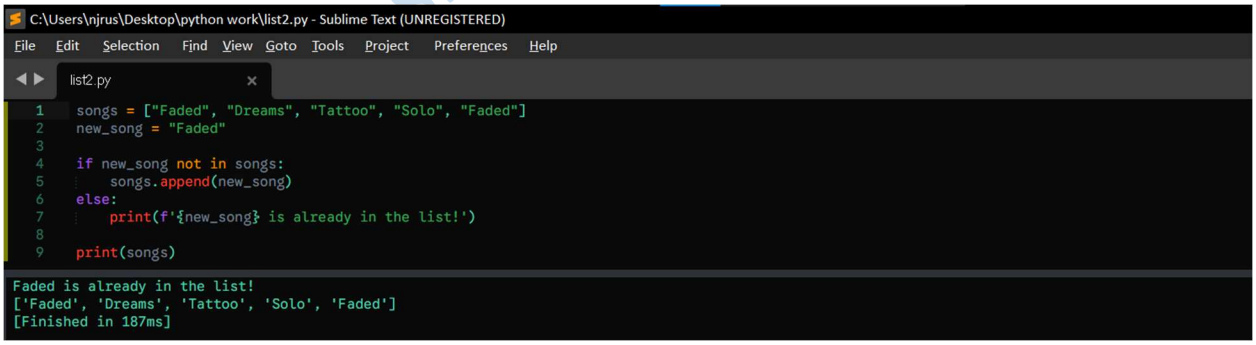
Membership checks are especially useful when a program needs to make a decision based on whether an item is already present. For example, when adding a song to a playlist, you might want to avoid adding it twice.




Python checks if the value of new_song is missing from the songs list. Since Faded is not currently in the list, this check evaluates to True, allowing Python to run the indented line of code where append adds Faded to the very end of the list. If new_song was already inside the list, the condition would evaluate to False and the code inside the block would be skipped entirely.



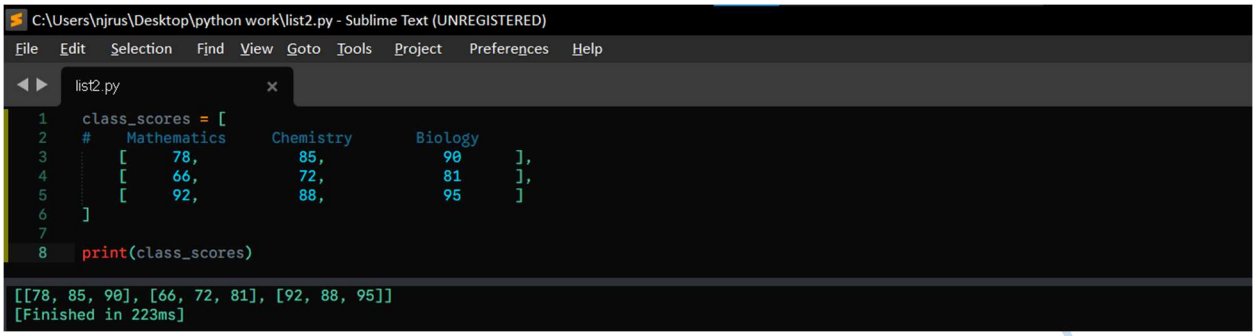

Keep in mind though that it only prevents new duplicates from being added and will not clean up any duplicates that might already exist in the list.




## 3. Nested Lists

A nested list is a list that contains other lists as its items. Nested lists can represent grouped data, such as rows in a table, game boards, or groups of related values.

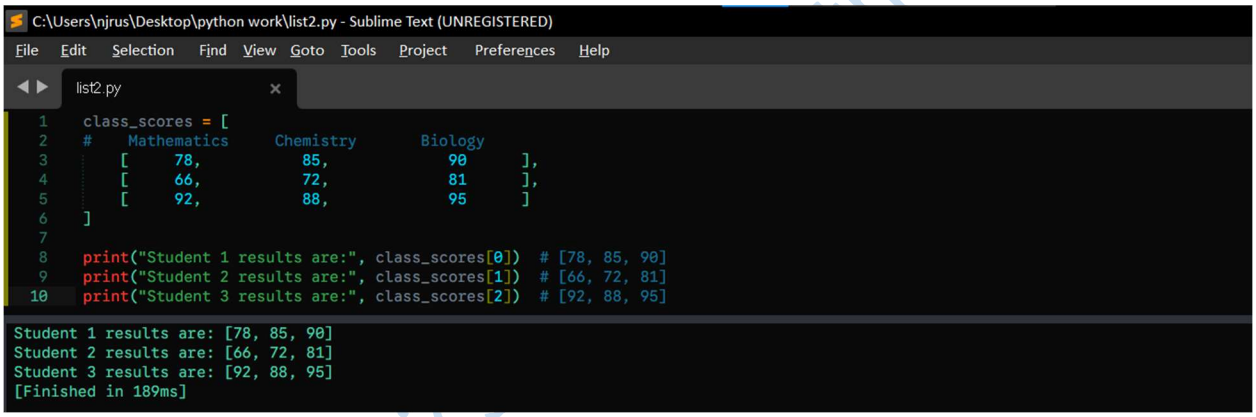
### Creating a nested list




Here, class_scores contains three inner lists. Each inner list could represent one student's scores across three tests. The outer list contains three items, and each item is itself a list.

### Accessing an inner list

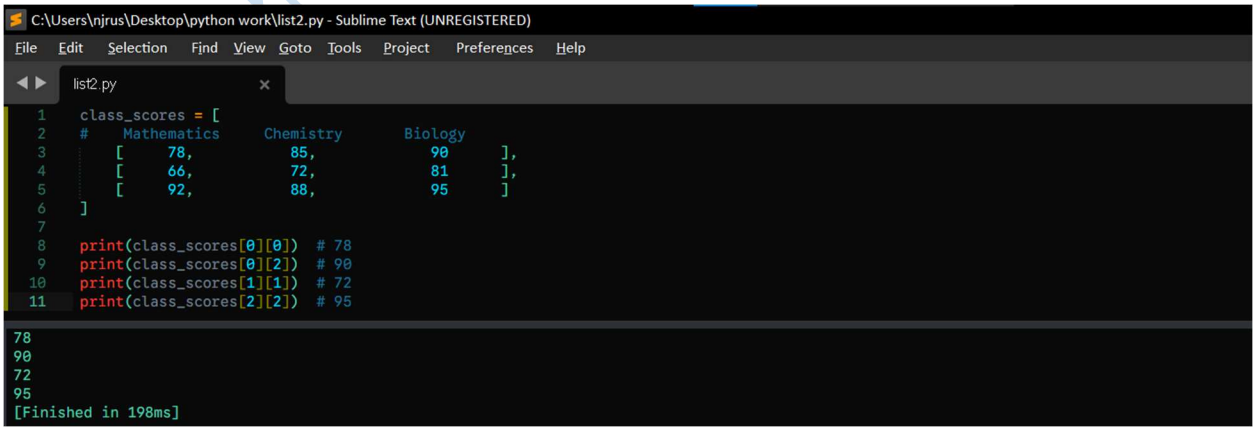
One index selects an inner list. Remember that indexing starts at 0.




class_scores[index] selects a list inside the outer list, not the first number inside it.

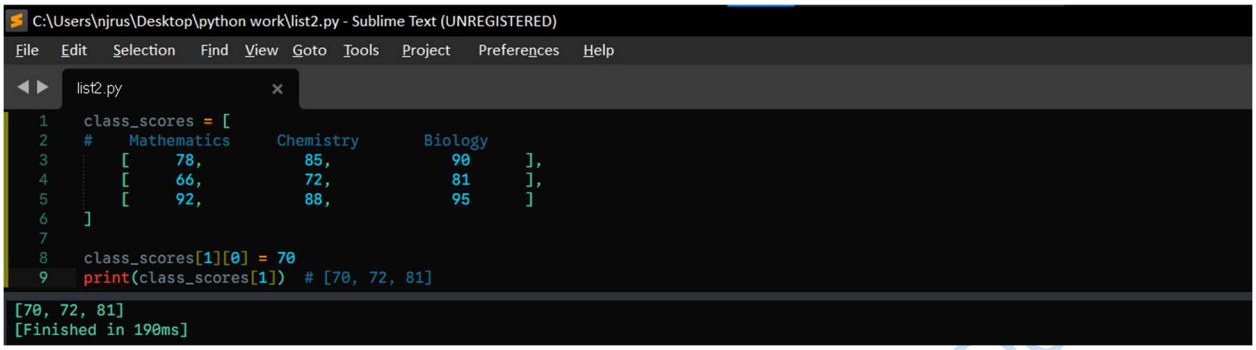
### Accessing an item inside an inner list

Use two indexes: the first selects an inner list, and the second selects an item inside it.



### Changing values in a nested list

You can also make updates to nested lists.




The two-index pattern lets you update an item inside an inner list.

From Basics to Junior Python D